Submission Worksheet

CLICK TO GRADE

https://learn.ethereallab.app/assignment/IT114-005-F2024/it114-milestone-2-chatroom-2024-m24/grade/bna24

Course: IT114-005-F2024
Assigment: [IT114] Milestone 2 Chatroom 2024 (M24)
Student: Bakr A. (bna24)

Submissions:

Submission Selection

1 Submission [submitted] 11/9/2024 11:24:16 PM

Instructions

COLLAPSE

1. Implement the Milestone 2 features from the project's proposal document: https://docs.google.com/document/d/1ONmvEvel97GTFPGfVwwQC96xSsobbSbk56145XizQG4/view
2. Make sure you add your ucid/date as code comments where code changes are done
3. All code changes should reach the Milestone2 branch
4. Create a pull request from Milestone2 to main and keep it open until you get the output PDF from this assignment.
5. Gather the evidence of feature completion based on the below tasks.
6. Once finished, get the output PDF and copy/move it to your repository folder on your local machine.
7. Run the necessary git add, commit, and push steps to move it to GitHub
8. Complete the pull request that was opened earlier
9. Upload the same output PDF to Canvas

Branch name: Milestone2

Group

Group: Payloads
Tasks: 2
Points: 2

COLLAPSE

Task

100%

Group: Payloads
Task #1: Base Payload Class
Weight: ~50%
Points: ~1.00

COLLAPSE

Details:
All code screenshots must have ucid/date visible.

Columns: 1

Sub-Task

100%

Group: Payloads
Task #1: Base Payload Class
Sub Task #1: Show screenshot of the Payload.java

Task Screenshots

Gallery Style: 2 Columns

4 2 1

Base Payload class

Caption(s) (required) ✓
Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Briefly explain the purpose of each property and serialization

Response:

The Payload class is used to store information for communication between the client and server. It includes payloadType (the type of command), clientId , and message . Serialization allows these objects to be converted into a format that can be sent over a network and later turned back into their original form.

Sub-Task

100%

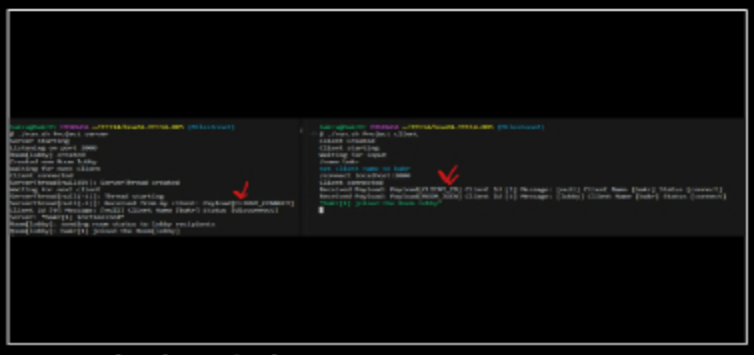
Group: Payloads
Task #1: Base Payload Class
Sub Task #2: Show screenshot examples of the terminal output for base Payload objects

Task Screenshots

Task Screenshots

Gallery Style: 2 Columns

4 2 1



BasePayload terminal output

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

End of Task 1

Task

100%

Group: Payloads
Task #2: RollPayload Class
Weight: ~50%
Points: ~1.00

COLLAPSE

Details:
All code screenshots must have ucid/date visible.

Columns: 1

Sub-Task

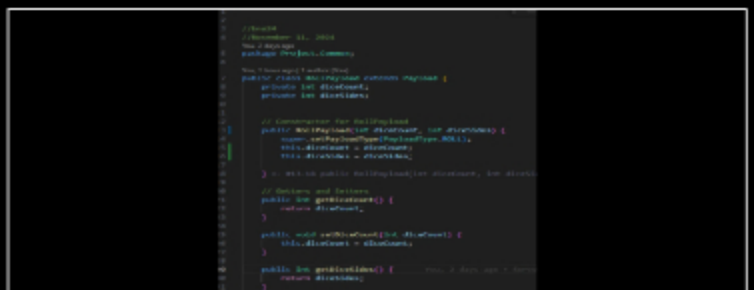
100%

Group: Payloads
Task #2: RollPayload Class
Sub Task #1: Show screenshot of the RollPayload.java (or equivalent)

Task Screenshots

Gallery Style: 2 Columns

4 2 1



RollPayload.java

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Briefly explain the purpose of each property

Response:

The RollPayload class has properties that define the specifics of a roll command: diceCount sets the number of dice to roll, and diceSides specifies the sides on each die. The payloadType marks this payload as a roll command, while clientId (which is inherited) identifies the client making the request.

Sub-Task

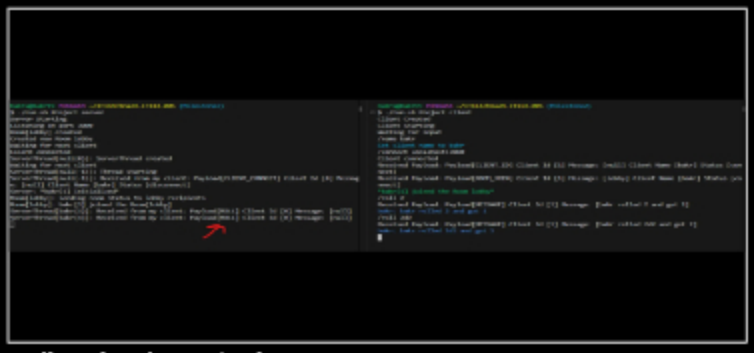
100%

Group: Payloads
Task #2: RollPayload Class
Sub Task #2: Show screenshot examples of the terminal output for base RollPayload objects

Task Screenshots

Gallery Style: 2 Columns

4 2 1



RollPayload terminal

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

End of Task 2

End of Group: Payloads
Task Status: 2/2

Group

100%

Group: Client Commands
Tasks: 2
Points: 4

COLLAPSE

Task

100%

Group: Client Commands
Task #1: Roll Command
Weight: ~50%
Points: ~2.00

COLLAPSE

Details:

All code screenshots must have ucid/date visible.
Any output screenshots must have at least 3 connected clients able to see the output.
All commands must show who triggered it, what they did (specifically) and what the outcome was.

Columns: 1

Sub-Task

100%

Group: Client Commands
Task #1: Roll Command
Sub Task #1: Show the client side code for handling /roll #

Task Screenshots

Gallery Style: 2 Columns

4 2 1

Client side /roll # format

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Briefly explain the logic

Response:

The client listens for a /roll # or /roll #d# command, parses the input to determine the number of dice (diceCount) and sides per die (diceSides), and creates a RollPayload object with this information. For a single roll (/roll #), it sets diceCount to 1 and diceSides to the specified number. for multiple dice (/roll #d#), it uses the values directly. The RollPayload is then sent to the server for processing, which generates and broadcasts the roll result to all clients.

Sub-Task

100%

Group: Client Commands
Task #1: Roll Command
Sub Task #2: Show the output of a few examples of /roll # (related payload output should be visible)

Task Screenshots

Gallery Style: 2 Columns

4 2 1

/roll #

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task 100%

Group: Client Commands
Task #1: Roll Command
Sub Task #3: Show the client side code for handling /roll #d# (related payload output should be visible)

Task Screenshots

Gallery Style: 2 Columns

4 2 1

/roll #d# format

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Briefly explain the logic

Response:

The client detects the /roll #d# command, parses it to extract the number of dice (diceCount) and sides per die (diceSides), then creates a RollPayload with this information. It sends this payload to the server, which processes the roll, calculates the total result, and sends the outcome back to the client. The client then displays the formatted result

Sub-Task

Group: Client Commands
Task #1: Roll Command

100%
Sub Task #4: Show the output of a few examples of /roll #d#

Task Screenshots

Gallery Style: 2 Columns

4 2 1

/roll #d#

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

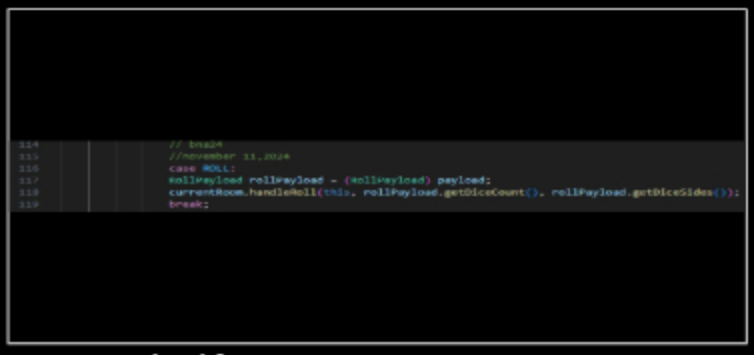
100%

Group: Client Commands
Task #1: Roll Command
Sub Task #5: Show the ServerThread code receiving the RollPayload

Task Screenshots

Gallery Style: 2 Columns

4 2 1



processPayload for ROLL

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Briefly explain the logic

Response:

The ServerThread class handles incoming RollPayload objects by detecting the ROLL payload type in the processPayload method. When a RollPayload is received, it is cast to access the diceCount and diceSides values, then passed to currentRoom.handleRoll, which performs the roll calculation and broadcasts the result to all clients in the room.

Sub-Task

100%

Group: Client Commands
Task #1: Roll Command
Sub Task #6: Show the Room code that processes both Rolls and sends the response

Task Screenshots

Gallery Style: 2 Columns

4 2 1

handleROLL

broadcastMessage

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Briefly explain the logic

Response:

The handleRoll method in the Room class processes roll commands by generating either a single dice roll or summing multiple rolls, based on the client's input. It creates a result message with the client's name and the total roll outcome. This message is then broadcast to all clients in the room using the broadcastMessage method, which sends the result to each connected client, allowing everyone to see the roll result.

End of Task 1

Task

100%

Group: Client Commands
Task #2: Flip Command
Weight: ~50%
Points: ~2.00

COLLAPSE

Columns: 1

Sub-Task

100%

Group: Client Commands
Task #2: Flip Command
Sub Task #1: Show the client side code for handling /flip

Task Screenshots

Gallery Style: 2 Columns

4 2 1

```
200                 //bna24
201                 //November 11, 2024
202                 case FLIP:
203                     sendFlipCommand();
204                     wasCommand = true;
205                     break;
```

Case FLIP

```
281     * bna24
282     * November 11, 2024
283     */
284    private void sendFlipCommand() {
285        FlipPayLoad flipPayload  = new FlipPayLoad();
286        send(flipPayload );
287    }
```

sendFlipCommand

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Briefly explain the logic

Response:

The /flip command is detected, and a FlipPayload is created and sent to the server. The server will then process the coin flip and return the result to the client.

Sub-Task

100%

Group: Client Commands

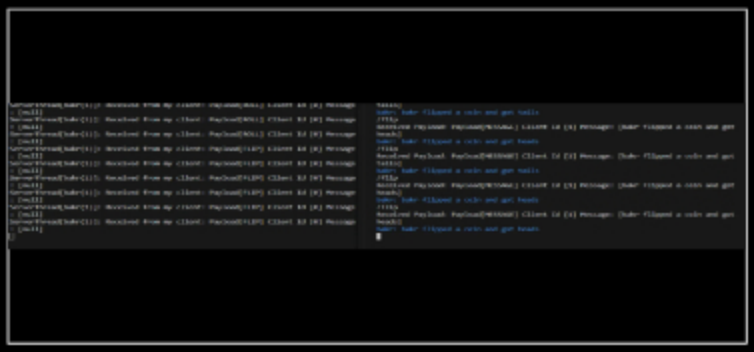
Task #2: Flip Command

Sub Task #2: Show the output of a few examples of /flip (related payload output should be visible)

Task Screenshots

Gallery Style: 2 Columns

4 2 1



/flip terminal output

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

End of Task 2

End of Group: Client Commands

Task Status: 2/2

Group

100%

Group: Text Formatting
Tasks: 1
Points: 3

COLLAPSE

Task

100%

Group: Text Formatting
Task #1: Text Formatting
Weight: ~100%
Points: ~3.00

COLLAPSE

Details:

All code screenshots must have ucid/date visible.
Any output screenshots must have at least 3 connected clients able to see the output.
Note: Having the user type out html tags is not valid for this feature, instead treat it like WhatsApp, Discord, Markdown, etc

Columns: 1

Sub-Task

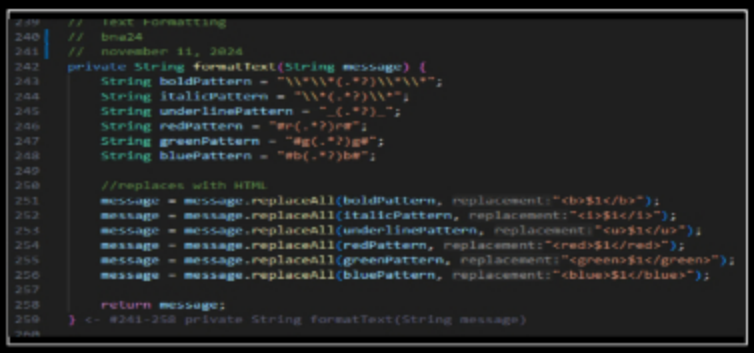
100%

Group: Text Formatting
Task #1: Text Formatting
Sub Task #1: Show the code related to processing the special characters for bold, italic, underline, and colors, and converting them to other characters (should be in Room.java)

Task Screenshots

Gallery Style: 2 Columns

4 2 1



```
//  Text Formatting
//  bna24
//  November 11, 2024
private String formatText(String message) {
    String boldPattern = "\\*\\*(.*?)\\*\\*";
    String italicPattern = "\\*(.*?)\\*";
    String underlinePattern = "_(.*?)_";
    String redPattern = "#r(.*?)r#";
    String greenPattern = "#g(.*?)g#";
    String bluePattern = "#b(.*?)b#";

    //replaces with HTML
    message = message.replaceAll(boldPattern, replacement:"<b>$1</b>");
    message = message.replaceAll(italicPattern, replacement:"<i>$1</i>");
    message = message.replaceAll(underlinePattern, replacement:"<u>$1</u>");
    message = message.replaceAll(redPattern, replacement:"<red>$1</red>");
    message = message.replaceAll(greenPattern, replacement:"<green>$1</green>");
    message = message.replaceAll(bluePattern, replacement:"<blue>$1</blue>");

    return message;
} <- #241-258 private String formatText(String message)
```

Code for special characters

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Briefly explain how it works and the choices of the placeholder characters and the result characters

Response:

The formatText method converts special characters in a message to apply formatting like bold, italic, underline, and colors. It uses placeholders such as **bold** for bold, *italic* for italic, *underline* for underline, and #r red r# for color (red,

green, blue). These placeholders are then replaced with HTML tags (`, , _`) ***for clients to display the text consistently.***

Sub-Task

100%

Group: Text Formatting
Task #1: Text Formatting
Sub Task #2: Show examples of each: bold, italic, underline, colors (red, green, blue), and combination of bold, italic, underline and a color

Task Screenshots

Gallery Style: 2 Columns

4 2 1

Terminal output

Caption(s) (required) ✓
Caption Hint: *Describe/highlight what's being shown*

End of Task 1

End of Group: Text Formatting
Task Status: 1/1

Group

100%

Group: Misc
Tasks: 3
Points: 1

COLLAPSE

Task

100%

Group: Misc
Task #1: Add the pull request link for the branch
Weight: ~33%
Points: ~0.33

COLLAPSE

Details:
Note: the link should end with /pull/#

Task URLs

URL #1
https://github.com/BakrAbushaar/bna24-IT114-005/5

URL
https://github.com/BakrAbushaar/bna24-IT114-0

End of Task 1

Task

100%

Group: Misc
Task #2: Talk about any issues or learnings during this assignment
Weight: ~33%
Points: ~0.33

COLLAPSE

Task Response Prompt

Response:

I had issues with serverthread not handling Roll properly and I didnt call a method in room that did the roll logic.

End of Task 2

Task

100%

Group: Misc
Task #3: WakaTime Screenshot
Weight: ~33%
Points: ~0.33

COLLAPSE

Details:
Grab a snippet showing the approximate time involved that clearly shows your repository. The duration isn't considered for grading, but there should be some time involved

Task Screenshots

Gallery Style: 2 Columns

4 2 1

Waka Time

End of Task 3

End of Group: Misc
Task Status: 3/3

End of Assignment